



MAnager for SoC Test

DEVELOPER Manual

Revision History

25/05/2023: Created



Index

1	Introduction	6
2	The MAST Kernel	6
2.1	MAST Execution Model	6
2.1.1	Thread handling and 1687 PDL	6
2.1.2	New Synchronisation model	6
2.1.3	Register Identification	7
2.2	The Execution Cycle	7
2.2.1	Configuration	8
2.2.2	Preconditioning	8
2.2.3	Data Cycle	9
2.2.4	Postconditioning	9
2.3	Test Data In Vector Composition	10
2.4	Serial Vector Format (SVF) usage	13
2.5	RTL cosimulation	14
3	Circuit Model	15
3.1	Introduction	15
3.2	General architecture	15
3.3	Common fields and methods	15
3.4	The REG node	16
3.4.1	Structural Description	16
3.4.2	Functional Description	17
3.4.1	Notable errors to be checked	18
3.5	The CHAIN node	18
3.5.1	Structural Description	18
3.5.2	Functional Description	18
3.5.3	Notable errors to be checked	18
3.6	The LINKER node	18



3.6.1	Structural Description.....	19
3.6.2	Functional Description.....	19
3.6.3	Notable errors to be checked.....	20
3.7	The ACCESS_INTERFACE node	20
3.7.1	Functional Description.....	21
3.7.2	Notable errors to be checked.....	21
3.8	The AI_TRANSLATOR node.....	21
3.8.1	Structural Description.....	21
3.8.2	Functional Description.....	21
3.8.3	Notable errors to be checked.....	21
3.9	Internal Representation and Data Endianness	21
4	Software considerations	24
4.1	Actual Status	24
Annex A	: Circuit Model examples.....	25
1.	High level usage	25
2.	Generic Thread Operation	25
3.	1149.1 TAP/Access Link	26
4.	AccessInterface	27
5.	1687 Segment Insertion Bit (SIB)	31
6.	Multiple Insertion Bit (BIT).....	34
7.	1500 Wrapper	38
8.	Multiple chains.....	40
9.	Daisy Chaining.....	41
10.	BSCAN2 Boundary Scan Linker.....	43
11.	I2C Access Interface/ P1687.1	45
12.	Mixed I2C/JTAG Access Link (Brocade).....	48



References.....	51
-----------------	----



Figure Index

FIGURE 1 EXAMPLE SYSTEM	10
FIGURE 2 CIRCUIT MODEL OF FIGURE 1 AND RESULT OF POSTORDER TRANSVERSAL	11
FIGURE 3 DETAILS OF A SCAN CYCLE (OUTPUT)	12
FIGURE 4 DETAILS OF A SCAN CYCLE (INPUT)	13
FIGURE 5 USAGE OF DEPTH AND BREADTH POINTERS	16
FIGURE 6 BIG ENDIAN BIT ORDERING	22
FIGURE 7 BIG ENDIAN BYTE ORDERING	22
FIGURE 8 PARTITIONING OF DATA FIELDS NOT MULTIPLE OF 8	22
FIGURE 9 SYSTEM MODEL OF A 1149.1 ACCESSLINK	26
FIGURE 10 ACCESSINTERFACE EXAMPLE	27
FIGURE 11 MAST SYSTEM MODEL FOR FIGURE 10	27
FIGURE 12 1687 EXAMPLE	32
FIGURE 13 MAST SYSTEM MODEL FOR FIGURE 11	32
FIGURE 14 MAST SYSTEM MODEL FOR THE MIB EXAMPLE	35
FIGURE 15 EXAMPLE OF A 1500 WRAPPER	38
FIGURE 16 MAST SYSTEM MODEL FOR FIGURE 15	39
FIGURE 17 DAISY-CHAINING OF JTAG TAPS	41
FIGURE 18 MAST SYSTEM MODEL FOR A 3-TAP DAISY CHAIN	42
FIGURE 19 BSCAN2 LINKER, FROM LATTICE DOCUMENTATION	43
FIGURE 20 MAST SYSTEM MODEL FOR THE BSCAN2 EXAMPLE	44
FIGURE 21 MAST SYSTEM MODEL FOR THE 1687.1 EXAMPLE	46
FIGURE 22 BROCADE JTAG MUX (FROM THE DATASHEET)	49
FIGURE 23 MAST SYSTEM MODEL FOR THE BROCADE MUX	49



1 Introduction

This document details the implementation of the architecture described in papers [1] and [2], and it is supposed to be an introduction for the developer who wishes to understand the source code and eventually propose modifications or extend it.

As such this document is a constant WORK IN PROGRESS and it is not guaranteed to be 100% synchronized with the repository's latest revision. For updated details of the source code, please refer to the Doxygen documentation.

2 The MAST Kernel

2.1 MAST Execution Model

2.1.1 Thread handling and 1687 PDL

The MAST execution model implies that threads post I/O requests to the manager when they need access to their respective register. This means that threads need to go to a sleep state and get woken up by the Manager when their data is ready.

To be compliant with 1687 PDL, MAST must present an API compliant to the iApply cycle:

- "iWrite(idf,data)" pushes "data" from the thread to the System Model (SM) by modifying the "seq_to_send" field of node 'idf' (see 3.4). It does not directly causes a data cycle and thread execution is not stopped.
 - NB: many realizations of iWrite will be needed depending on the type of "data"
- "iGet(idf,*data)" retrieves the data received during the last iApply cycle and stored in the "last_received_seq" field of node 'idf' (see 3.4). It does not directly causes a data cycle and thread execution is not stopped.
- "iApply" is the explicit synchronization request. It generates a "Read-Capture Update" cycle that causes the data in the System Model being updated. It causes the thread to go into a "sleep" state and the "pending_threads" flag of the node incremented by 1 (see 3.4).

2.1.2 New Synchronisation model

Nodes are woken up after the System Model data cycle phase (cfr: next section). In the current implementation, due to legacy issues, each thread actually locks the System Model. Also, an iApply is relative to a specific node, as it needs to change the status of its mutex. This is not optimal for several reasons, the more important being:

- There is actually no need of locking the SM. This only degrades performances

- In 1687, iApply is a system-wide operation. A given PDL code could iWrite multiple registers before calling an iApply, and it should resume operation only when all iWrites have been taken into account, which might take multiple cycles if one or more of the registers are behind, for instance, a closed SIB.

The standard is not extremely clear about this part, but in order to respect the minimum the synchronization should mark as following:

- Each time a thread executes an iWrite, this should be recorded in a thread-specific queue.
- All threads are blocked on the same mutex/semaphore, either global or defined at root of the System Model (I there need for a specific node?).
- During the data-write cycle (see section 2.2.3), each time an iWrite is executed the corresponding node is removed from the queue
- When the queue becomes empty, the node can be woken up
 - Alternative, all nodes with empty queues can be woken up once the data-write cycle is finished

2.1.3 Register Identification

In PDL registers are identified with their name, which must correspond to the one declared in the associated ICL. Some provisions are given for the cases when the same ICL/PDL pair is instantiated multiple times to assure unique access, the iUseProcNameSpace and iPrefix commands, which imply a knowledge of the ICL structure because they work as modifiers of the register names (ex: by defining a prefix, so for instance “reg1” stands for “module.instrument.reg1”.

In MAST, each register has a unique identifier which can be given the thread as input parameters (it is done in all the examples of Annex A). Optionally, some behavior similar to PDL might be defined in the API, for instance by the usage of C++ “define” or “namespace”.

2.2 The Execution Cycle

The heart of MAST is the execution cycle, which is based on a 4 stage execution:

- Execution is triggered by the SM having some “pending” nodes
 - A full-tree transversal is triggered to configure the SM;
 - An active-tree transversal takes care of preconditioning
 - An active-tree transversal executes the data cycle. This results in one or more AccessInterface operations (see 3.7) that will cause the SM to be updated
 - An active-tree transversal takes care of postconditioning

- If any pending flag remains, loop to step 1.

NB: a “full tree” is a traditional transversal where recursion is done on all derivations. An “active-tree” transversal means that recursion of a derivation of a LINKER node is done only if the “is_active” method returns true.

In the current code there are some additional phases (ex: “dry_run”) used to display debug information.

2.2.1 Configuration

A Configuration can be triggered by many reasons by the Manager. Typical examples are either the triggering of a watchdog or having each node the System Model pending (i.e. “fullpending” option). There are two types of pending flags, as detailed in section 3.3, but their effect is the same in Configuration terms.

Configuration is a depth-first transversal of the full tree aimed at determining a suitable configuration to satisfy the pending nodes through a bottom-up approach. The transversal first recurses until reaching leaf nodes, and then:

- If the node is a REGISTER, returns its “pending” status;
- If the node is a CHAIN, returns a logic “OR” of the pending from its derivations
- If the node is an ACCESS_INTERFACE, returns a logic “OR” of the pending from its derivations
- If the node is an AI_TRANSLATOR, returns the pending status from its derivation
- If the node is a LINKER, a configuration action takes place. This action is based on the methods is_active/select/deselect: the idea is to check if a pending derivation “is_active”, and to “select” it if not. Non pending derivation might also be “deselected” to shorten the active scan chain.
 - The efficiency of the manager is directly correlated to this configuration algorithm, so it must be easily configurable and customizable (a plugin?).
 - A “selection”/“deselection” action might imply changing the values of one or more registers, causing the nodes to be marked as “pending_selection”

2.2.2 Preconditioning



Some nodes might require special conditioning operations prior to the main data cycle. It is the case, for instance, of AI_TRANSLATOR nodes, but it could also be the case for legacy nonstandard implementation. Preconditioning is a depth-first exploration of the active tree.

NB: only a minority of nodes demands preconditioning, but in the current implementation this phase always implies a tree transversal. A possible optimization could be triggering the operation only if at least one node requires it.

2.2.3 Data Cycle

The Data cycle is where communication with the System Under Test is done in three steps:

- An active-tree transversal is done to obtain the output vector. By construction, a depth-first transversal orders the registers in the right order. If the node has a “pending_thread” the “seq_to_send” field is used, otherwise it is the “bypass_seq”. This sequence is the “Data-read cycle”
- The action of the top-level ACCESS_INTERFACE causes the output vector to be sent and an input vector to be received
- The input vector is used to update the “last_received_seq” of each node traversed during the first step. In the current version nodes to be updated are stored in a dedicated structure, but this has no effect on the performances, in the final version this should be done by an active tree transversal the “Data-write cycle”

Threads locked on a node semaphore are woken up as soon as the node is updated and the “pending_threads” flag is cleared. “Pending_select” flags are cleared on nodes updated during a data cycle.

NB: in the current implementation, input and output vector are global variables. It would probably be more correct to make them local to a given AccessInterface

2.2.4 Postconditioning

Some nodes might require special conditioning operations after the main data cycle. It is the case, for instance, of AI_TRANSLATOR nodes, but it could also be the case for legacy nonstandard implementation. Postconditioning is a depth-first exploration of the active tree.

NB: only a minority of nodes demands postconditioning, but in the current implementation this phase always implies a tree transversal. A possible optimization could be triggering the operation only if at least one node requires it.

2.3 Test Data In Vector Composition

The vector to be scanned to the System Under Test TDI pin is composed thanks to a postorder transversal of the Circuit Model: the first nodes touched are the first to be scanned in, as shown in Figure 1 and Figure 2. To have a correct vector it is important to respect the bit ordering, regardless of the endianness of the host processor executing the manager.

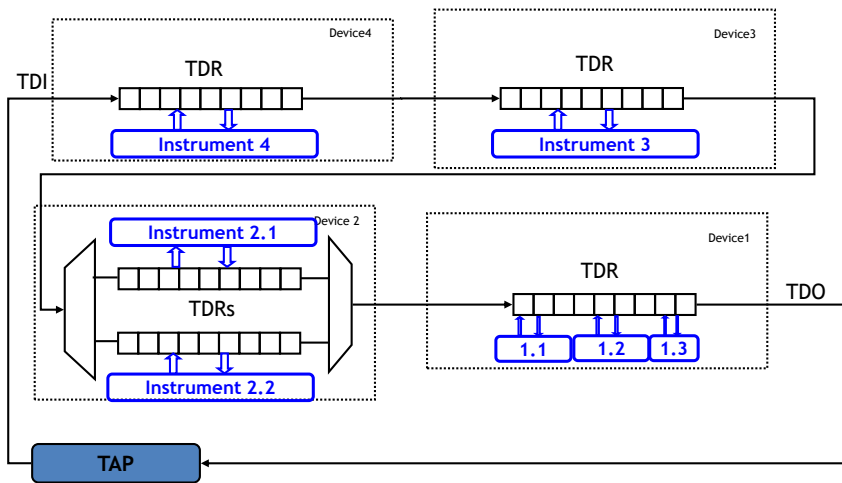


Figure 1 Example System

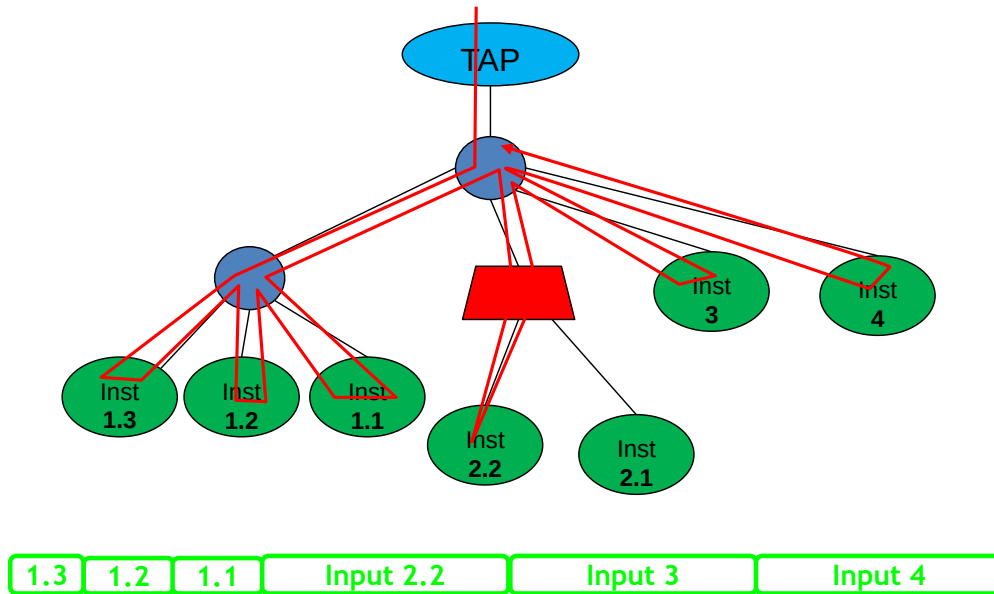


Figure 2 Circuit Model of Figure 1 and result of postorder transversal

The vector depicted at the bottom of Section 0 can be easily obtained by concatenating the data fields of the Circuit Model nodes. Thanks to the internal representation explained in section 3.9, the data can then be scanned following the natural order of the bit representation. Figure 3 details a scan operation from the output side (i.e. data sent to the SUT)

- The System Under Test (SUT) before the scan operation, composed by the concatenation of an 8-bit and a 16-bit register
- The corresponding Circuit Model with the new data resulting from iWrite operations. Please note how the internal representation conversion is done for “inst 1” by the iWrite function
- The vector obtained by concatenating the data fields of the circuit model.
- The vector actually scanned, from the least significant bit (bit 0 of the first byte) to the most significant (bit 7 of the last byte)
- The SUT after the scan operation. Please note how the data is aligned in the register respecting the original iWrite call bit ordering.

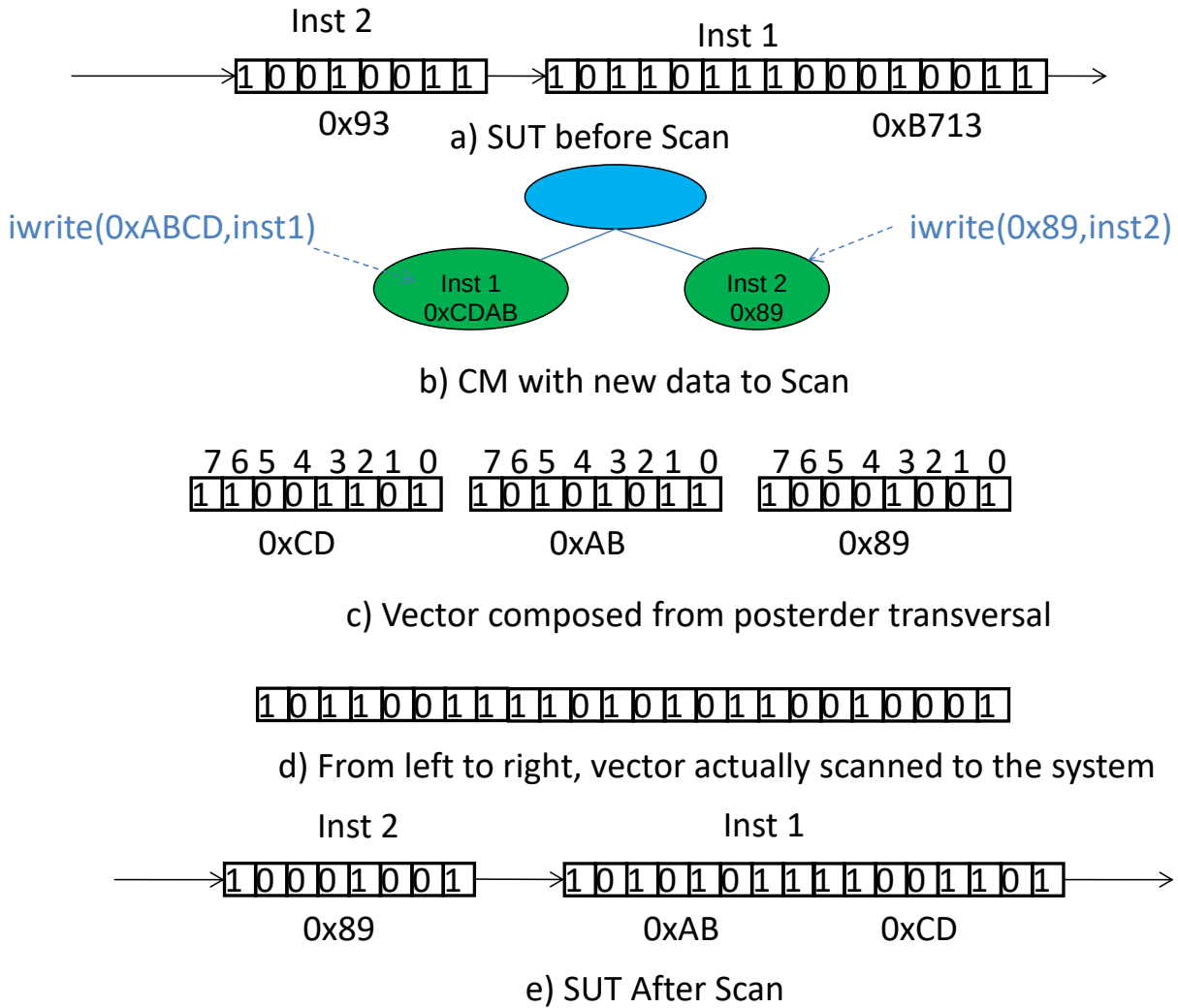


Figure 3 Details of a scan cycle (output)

Figure 4 details the same scan cycle, but from an input point of view (i.e. data received from the SUT)

- The state of the SUT before the scan operation
- The data received from the SUT, arranged from left (first bit received) to right (last bit received)
- The data of (b) arranged in the internal representation of the circuit model (Section 3.9).

- d. The Circuit Model, updated following a postorder transversal. Please note how the data returned by iGet is exactly in the same aligned as the in the SUT registers

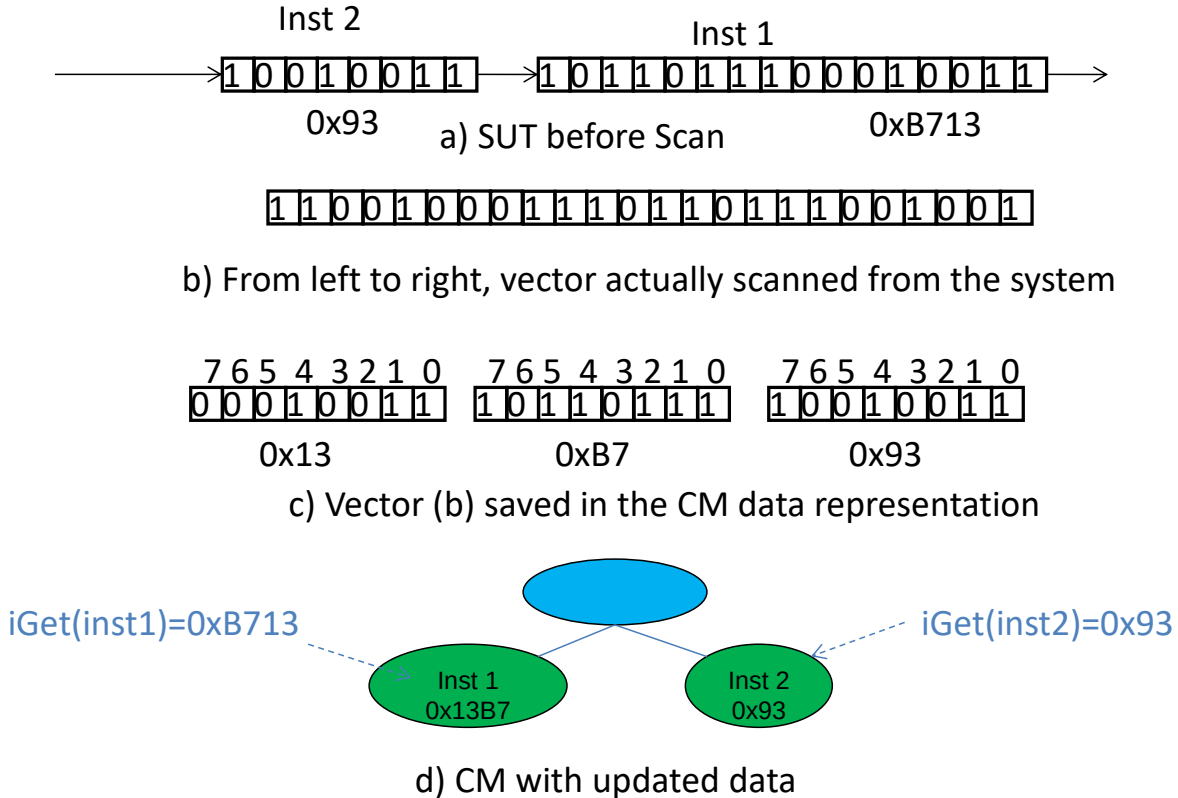


Figure 4 Details of a scan cycle (input)

2.4 Serial Vector Format (SVF) usage

Most USB-JTAG interfaces are used as SVF players to control the TAP. Even in more sophisticated approaches like openOCD, vectors follow SVF conventions (TO BE CONFIRMED!!!), so it is important to be

SVF data representation mandates vectors to be represented as hexadecimal strings, with the rightmost bit representing the first bit scanned to/received from the TAP. Is its therefore the opposite order as per Figure 3 d) and Figure 4 b).

As a consequence, between steps d) and e) of an Output Scan Cycle (Figure 3), it might be necessary to convert the binary data into an SVF string, which in fact corresponds with the scan chain in the SUT after the scan cycle (0x89ABCD in Figure 3).



On the other hand, during an Input Scan Cycle (Figure 4) there will probably be no need to pass through the binary representation of step b): it will be easier to directly convert the string received from the interface (0x93B713) into the CM representation of step c).

Please note that 1687 operations as shown in Figure 3-b and Figure 4-d use and SVF-like data representation.

NB: SVF representation is not optimal in terms of memory occupation, as each vector bit needs two memory bit (one hex value/byte => 4 vector bits/byte). Also, algorithms being executed on the test host will most probably use native data representation, so a conversion to SVF would be needed for each output operation (and similarly for input).

2.5 RTL cosimulation

MAST can be interfaced with Modelsim to obtain VHDL co-simulation. This is done in two steps:

- MAST be launched using the “RTL” flag.
 - o This causes the program to output the AccessInterface commands SIR and SDR to a text file in the ./to_RTL/ directory
 - o In RTL mode, MAST looks for data returned from the System Under Test in a text file in the ./from_RTL/, where binary data is written as an ‘0’ and ‘1’ ASCII characters
- In Modelsim, a specific top-level 1149.1 TAP must be used. This module implements both an SVF interpreter to execute commands issued from MAST and a binary log to return data.
- NB: The Modelsim “write” function for file-based I/O does not have a deterministic behavior: buffer flushes happen at unpredictable intervals, which might cause the co-simulation to stall. To solve this problem, the “MTI” libraries have been used to define a deterministic “write” (written in C) that flushes the buffer at the end of each call.

To obtain an interactive execution, both exchange files must be empty when launching MAST and the RTL simulation. Otherwise, MAST will start by reading the input vectors already in the “from_RTL”, while Modelsim will start by executing the SVF command already in the “to_RTL” directory. This behavior might be used to have scripted simulations (ex: execute a known set of SVF commands in Modelsim).

3 Circuit Model

3.1 Introduction

The role of the Circuit Model is to contain and organize the minimum information set needed for operating the System Under Test (SUT). This means:

- The size and content of each scan register
- The sequence of scan registers
- The hierarchical information
- The Access Interface

Each of these 4 pieces of information is represented as a Circuit Model element.

3.2 General architecture

The MAST Circuit Model is based on a tree structure: the root node represents the main Test Access Port (TAP) and the arborescence mirrors the hierarchy of SUT, and its internal state is synchronized with the SUT's. When a data cycle (scan) is required, a transversal is triggered on the tree. This transversal is dependent on the Circuit Model State, therefore allowing MAST to handle dynamic hierarchies like the ones supported by IEEE 1687. More details are given in Section **Erreur ! Source du renvoi introuvable.**

3.3 Common fields and methods

All nodes share the following fields and methods:

- An Unique Integer Identifier
- An ASCII name
- An enumerated type
- depth_next : pointer to the first node of the lower hierarchical level (if any). Represented by a dashed arrow in Figure 5
- breadth_next: pointer to the next node of the same hierarchical level (if any). Represented by a solid arrow in Figure 5
- append(): method to add a node as the last one of the lower hierarchical level.
- A generic "content"
- Pre_conditioning field and method: define optional conditioning (i.e. operations) to be executed before the data cycle
- Post_conditioning field and method: define optional conditioning (i.e. operations) to be executed after the data cycle

- An integer Priority, which might be used in the configuration phase

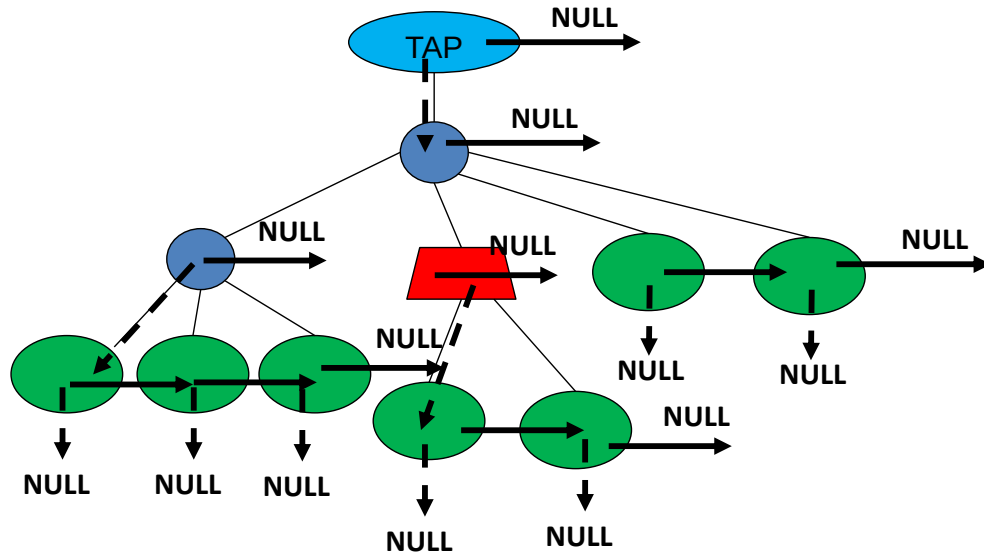


Figure 5 Usage of depth and breadth pointers

3.4 The REG node

The REG node is the base element for the MAST Circuit Model, and represents a Test Data Register.

3.4.1 Structural Description

It contains these fields:

- Fields describing the structure of the node
 - o tdr_len: the number of bits composing the Test Data Register
 - o tdr_bytes: the number of complete bytes composing the register (result of integer division $\text{tdr_len}/8$, to avoid having to compute it each time);
- Fields used for execution
 - o bypass_seq : the default bypass sequence, to be used when no iApply cycle has been defined on the register
 - o seq_to_send : Sequence to be sent during the next iApply cycle
 - o last_sent_seq: Sequence sent during the last iApply cycle.
- Fields used for checking expected values (not implemented yet)
 - o last_received_seq: sequence received during the last iApply cycle
 - o check_expected : if set to '1', it triggers causes a check of received vs expected data during the follow

- expected_seq: expected data sequence
 - mismatch: number of mismatches following IEEE 1687 rules
- Field used for CM configuration and execution
 - pending_threads : number of threads having executed an iApply on the TDR register
 - pending_select : set to '1' if the tdr value has been changed following a selection action (see section)
- Desired fields (not implemented yet):
 - To assure flexibility and forward compatibility, it should be possible to define new fields, for instance to express priority or to convey specific information (ex: power consumption)

It contains these methods:

- send_funct : method responsible for low-level delivery of scan data. By default, it appends the tdr value to the overall scan vector, as explained in Section 0.
- <TODO: method to receive data from last scanned vector, as of now implemented as the "extract_from_input_vector" function in module ll_access>

3.4.2 Functional Description

This node represents a JTAG register, and it has four functional roles:

- Describe the data access respecting the Capture-Shift-Update cycle: "last_received_seq" contains the data received FROM the SUT during the last access (Capture to save data from the SUT into the JTAG register + Shift to collect it through the TAP), while "seq_to_send" is the data to be sent TO the SUT (Shift to bring it from the TAP to the register + Update to apply the value to the SUT);
- Describe execution behavior according to 1687 PDL: "seq_to_send" is modified by iWrite, while iGet retrieves data from "last_received_seq". iRead (not implemented yet) asks for a comparison between the "expected_seq"ence and the data actually received in "last_received_seq"
- Check that the System Model is synchronized with the SUT by comparing the "last_sent_seq" with the "last_received_seq" (not implemented yet)
- Provide information to the System Model for execution: "pending_thread" means that a thread modified the "seq_to_send", while "pending select" means a "select/deselect" operation has been made (see Section 2.2.1) and the bypass sequence needs to be sent to the SUT.

3.4.1 Notable errors to be checked

- No node might be appended to a REGISTER, they must always be leaves

3.5 The CHAIN node

The CHAIN node allows nodes to be concatenated to replicate a scan chain composed of TDRs.

3.5.1 Structural Description

This node has no specific fields or methods

3.5.2 Functional Description

This node represents the concatenation of elements (register, ScanMuxes etc..) in the SUT through the cardinality of children nodes: node(i+1) follows node (i).

Please note that a CHAIN node with only one derivation has no functional effect on the execution, but it can be used to package some sub-system and exploit the “append” method to assure correct construction. For instance in 1500 example (Annex A. 7, Figure 16) a CHAIN node called “1500_wrapper” is defined, and all calls to “append” result in the new node being appended to the “WIR_Mux” node.

3.5.3 Notable errors to be checked

- None

3.6 The LINKER node

The linker node is responsible for modeling dynamic topologies, like for instance the ScanMux/SIB of 1687. Like the SUPER node it has some derivations stored as child, but they are not always available (i.e. “active”) for data access. The node therefore provides some fields and methods that the System Level Manager uses to handle configuration.

The principle of operation is simple: based on its status, the LINKER can determine if each of its derivations is “active”, and is therefore part of the active scan path. By overloading the four methods a LINKER node can represent any arbitrary dynamic element.

3.6.1 Structural Description

- Fields:
 - available_paths : maximum number of derivations that can be appended to the node
 - selection_table: selection_table[i] contains the value to select path i when i>0. selection_table[0] corresponds to no path selected
 - deselection_table: deselection_table[i] contains the value to de select path i when i>0. deselection_table[0] corresponds to no path selected
 - set_linker_info: generic information to be used to access the linker status
- Private Methods: (linker information)
 - Set_linker: method responsible for changing the status of the linker, based on the “set_linker_info” field
 - get_last_seq: returns the last sequence sent to the linker, i.e. its current status
 - get_next_seq: returns the sequence to send to the linker during the data cycle, i.e. its future status
- Public Methods: (linker usage)
 - is_active(n): returns 1 if derivation “n” is active
 - select(n): changes the linker status to make “n” active in the following data cycle
 - deselect(n): changes the linker status to make “n” inactive in the following data cycle

3.6.2 Functional Description

The LINKER node provides an abstraction and extension to the ScanMux: it represents a generic component that allows the dynamic selection of a set of segments, represented by its children. For this, it uses the triplet “is_active/select/deselect”: the execution and configuration algorithms and iterators (see Section 2.2) use only this triplet. Thanks to function overloading, MAST can therefore express any LINKER, regardless of its complexity. To allow for this extension, a generic “set_linker_info” is provided to store useful information, and a “set_linker” method to make the low-level operation.

In practical terms, almost all LINKERs used in our use cases (Annex A) follow the 1687 ScanMux rules: the LINKER itself is a multiplexer, whose control value comes from REGISTER. For SIB for instance the fields and methods are declared as follows:

- available_paths : 1
- selection_table: {‘0’,‘1’}. So, the only derivation is activated by selection_table[1]=‘1’
- deselection_table: {‘0’,‘0’}

- `set_linker_info`: the Unique Integer Identifier (UID) of the register controlling the SIB
- Private Methods:
 - `Set_linker`: this method changes the “bypass_seq” of the node identified by `set_linker_info` and sets its “pending_select” flag to 1;
 - `get_last_seq`: this method returns the “last_received_seq” of the node identified by `set_linker_info`
 - `get_next_seq`: this method returns the “seq_to_send” of the node identified by `set_linker_info`
- Public Methods:
 - `is_active(n)`: returns 1 if the value returned by `get_last_seq` == `selection_table[n]`
 - `select(n)`: calls `set_linker` with the value `selection_table[n]`
 - `deselect(n)`: calls `set_linker` with the value `deselection_table[n]`
 - NB: for all 3, n can only be 1

For a MIB, the only differences are the “available_paths” and the `selection_table/deselection_table` fields. By changing the values of the latter it is possible to express different codings (one-hot, binary, etc...). Modifications to the “is_active/select/deselect” can also model different behaviors (ex: MIBs where more than one segment can be selected). MAST provides wrappers to easily instantiate and configure notable LINKER configurations).

3.6.3 Notable errors to be checked

- Trying to append more than the “available paths”
- Trying to call “is_active/select/deselect” on an empty derivation

3.7 The ACCESS_INTERFACE node

The Access Interface (AI) node is responsible for representing physical interfaces to the internal registers represented by the previous node types. Structural Description

This is reflected by the following fields:

- `protocol_size`: number of primitives composing the protocol
- `n_derivations`: number of nodes (derivations) accessible through the AI.
- `optional_AI_data`: generic field for AI-related data
- `AI_protocol[protocol_size]`: array of the “protocol_size” primitives composing the protocol
- `AI_action[n_derivations]`: `AI_action[n]` contains the pointer to the function allowing access to the n-th derivation

The operation and usage of the ACCESS_INTERFACE node might be clearer by looking at the examples 3, 4, 10, 11 and 12 given in Annex A.



3.7.1 Functional Description

The AccessInterface has two roles:

- represent an Access Protocol, i.e. a set of Primitive operations available on the physical interface
- provide Actions to access the derivations based on the set of primitives

For a more detailed explanation, see ITC 2016 paper

3.7.2 Notable errors to be checked

<TBD>

3.8 The AI_TRANSLATOR node

<To Be Implemented Yet>

The role of the AI_TRANSLATOR is to model an adapter from an Access Interface to another, like for instance a I2C-JTAG adapter.

3.8.1 Structural Description

3.8.2 Functional Description

3.8.3 Notable errors to be checked

3.9 Internal Representation and Data Endianness

The data fields of the Circuit Model have two roles:

- Represent the hardware registers inside the System Under Test
- Act as interfaces to the algorithms being executed on the Host

These two roles can be in contradiction in terms of the internal data representation, as the Host processor will have its internal endianness, which might not be the same as in the target hardware. Endianness can also be a problem when composing the scan vectors, as detailed in Section 0. The internal representation of the Circuit Model is therefore chosen to optimize vector handling, and it is the role of the API methods to assure the conversion to the representation of the host circuit. Data is handled as following:

- At bit level, data is represented as classic Big Endian, as depicted in Figure 6

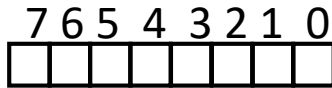


Figure 6 Big Endian Bit Ordering

- At byte level, Little Endian is used. So, for instance, the hex value 0xABCD would be stored as 0xCD 0xAB, as depicted in

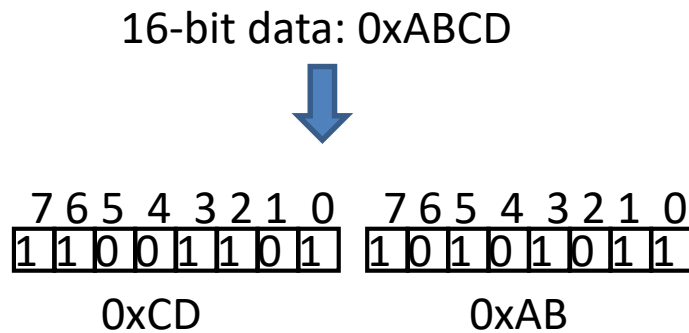


Figure 7 Big Endian Byte Ordering

- When the size of a data field is not an exact multiple of 8, byte partitioning starts from the least significant bit. Unused bits are left as “don’t” care, without any filling. An example is shown in

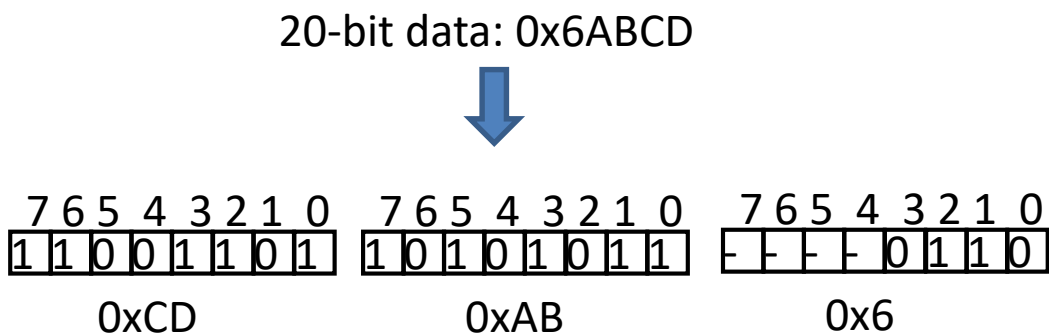


Figure 8 Partitioning of data fields not multiple of 8



Please note that this is the same convention used for traditional x86 architectures, so in most PC-based host there will be no need of conversion for basic types like integers. No guarantee is given for data whose size is bigger than 4 bytes.



4 Software considerations

4.1 Actual Status

MAST is actually implemented as a pure C object, where some Object-Oriented features are manually implemented. This results in an efficient but sometimes difficult-to-read code, and with a significant amount of code replication, notably when traversing trees.

Only the most important error detection is done, as a general rule the current code base supposes a correct usage and is not tolerant to erroneous calls.

Most examples pass “valgrind” with no errors, but no extensive checks have been done. Note that often valgrind errors come from the examples themselves rather than from the code.

The file organization is as follows:

- root directory
 - o Makefile, README and similar files
- Each of these modules has a .h header in /include/ and a source file in /src/
 - o CM : definition of basic blocks and constants for the Circuit Model
 - o SL_Manager: Implementation of the System Level Manager
 - o SLM_Interface: functions for threads to interface with the System Level Manager
 - o threadsafe-stdio: thread-safe definitions for printf (based on Stevens)
 - o ll_access: low-level access (ex: bit and byte-wise) operations
 - o 1149: CM representation of elements from the IEEE 1149.1 standard
 - o P1687: CM representation of elements from the IEEE 1687 standard
 - o I2C: CM representation of elements from the I2C Access Interface
 - o brocade: CM representation of the Brocade Boundary Scan Linker (I2C+JTAG)
- /include/ : headers without a corresponding source file
 - o errocodes.h : definition of error codes (incomplete)
- /my_testcases/ : use cases
 - o ...
- /to_RTL/: exchange directory for sending data to the Modelsim simulator
 - o ...
- /from_RTL/: exchange directory receiving data from the Modelsim simulator
 - o ...
- /outputs/, /references/: storage of reference outputs (outdated)
- /desktop_build/: build directory for PC target
- /vhd/ : VHDL files to simulate the back-end in Modelsim (outdated, updated sources in CIME account)

Annex A : CIRCUIT MODEL EXAMPLES

1. HIGH LEVEL USAGE

All use cases are constructed to be validated through logs: each MAST module generates ASCII messages printed on the screen. This output can then be redirected to a file for offline analysis or processed using grep: each module prepends a specific prefix to its messages, so it is easy to separate each output. Notable usages are:

- grep “(CM)” for the Circuit Model module debug messages
- grep “thread 0” for the debug messages issues by thread 0 (see next section)
- grep S.R to extract the SVF operations generated by the 1149.1 AccessInterface. NB: the I2C AI prepends “S2R” to its commands so it can be caught with the same syntax.

In the rest of the section, the MAST System Model is shown through screenshots of its gml dump.

LEGENDE CODE COULEUR

2. GENERIC THREAD OPERATION

To have a consistent test base, all tests share the same instrument algorithm: a thread that makes randomly-timed accesses to its segment. The bit sequence to be shifted is incremented each time, starting from a value given at launch time (usually, the identifier of the segment). To check 1687 compliance, each thread executed a variable number of times the same cycle: two iWrites (the latter overwriting the first), an iApply and an iGet. Correct operation can be certified by observing in the SVF operations the right sequences.

In each example a thread is associated with a register, and a rendez-vous is established thanks to a thread “join” for all threads. So, the example can finish only if each thread has correctly finished execution.

All examples share the same structure:

- A “chain_conf_xxx” function which uses the API to build the System Model and saves the identifiers of the Register nodes

- A “run_test” that uses the identifiers saved by the “chain_conf_xxx” to configure, launch the threads and set the rendez-vous
- Each example has some “define” that can be used to customize it.

The actual thread code is in the “testcase_tools.c” file which must be included by each example

3. 1149.1 TAP/ACCESS LINK

All examples have as root level one or more 1149.1 Test Access Port (TAP), called AccessLink in 1687 terms. This is modeled as an AccessInterface node having two derivations, the IR and DR branch, accessed by the SIR and SDR operations respectively. On the DR branch, a LINKER handles chain selection based on the value of the IR register. A wrapper function is given to instantiate the set, depicted in Figure 9

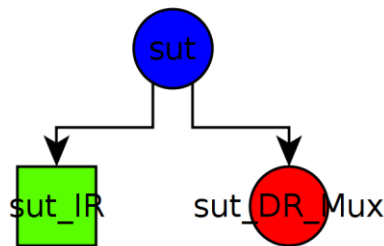


Figure 9 System Model of a 1149.1 AccessLink

If no command line parameter is given, the AccessInterface is configured in a loopback emulated mode: the output vectors computed by MAST are copied to the input buffer. If the “RTL” option is given, input vectors are read from the “from_RTL” directory (see section 2.5).

NOTES

- the “append’ method of the root “SUT” node is overloaded to have sub-chains appended to the LINKER

- IR coding puts a limit to the selectable derivation of the DR_mux, but no check is done by “append”. Extra nodes are appended with no error, but they won’t be accessible
- In the current status, the “RTL” option is given as an input parameter for the MAST “scheduler” module

4. ACCESSINTERFACE

This corresponds to the simplest JTAG systems, depicted in Figure 10: a TAP with “n” chains (2 in the example), that can be selected through the IR register. JTAG compliance asks for a 1-bit bypass register to be added.

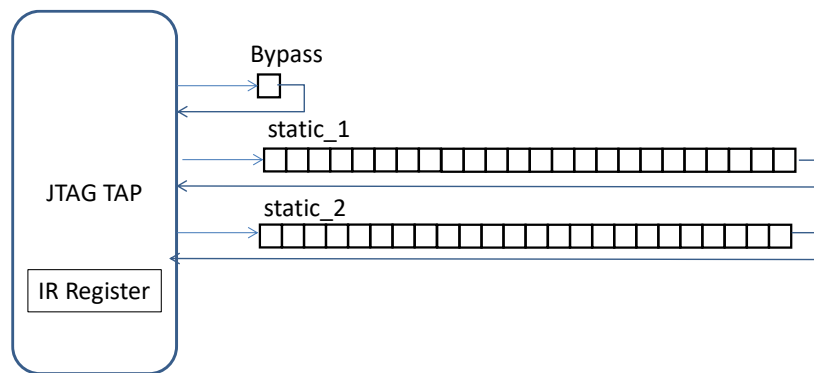


Figure 10 AccessInterface Example

In MAST, the System Model representing this example is depicted in Figure 11.

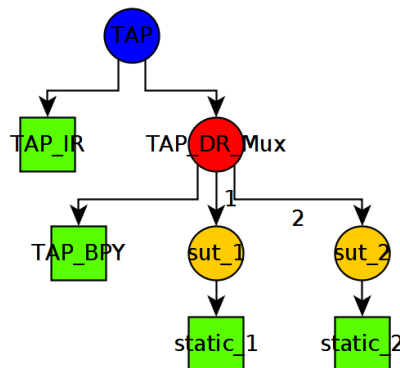


Figure 11 MAST System Model for Figure 10

Threads are launched on the Registers “Static_1” and “Static_2”, each performing 4 iterations. The resulting operation is therefore a succession of “SIR’ operations selecting one of the



registers and of “SDR” operations to shift the vectors. The actual succession depends on the timing characteristics of each thread. In an exemplary execution, the commands `grep “thread 0”` and `grep “thread 1”` return this log.

```
[thread 0] spawned at 1460539032.991667 s for node 5
[thread 0] Starting cycle 0 at 1460539032.991680 s
[thread 0] making an iWrite
[thread 0] Executed at 1460539032.991740 s
[thread 0] Making an iWrite (precedent value overwritten) ...
[thread 0] Byte Value to be shifted at node 5 is 0x05
[thread 0] Calling an iApply ...
[thread 0] iApply finished at 1460539033.015146 s
[thread 0] Starting cycle 1 at 1460539033.015153 s
[thread 0] making an iWrite
[thread 0] Executed at 1460539033.015182 s
[thread 0] Making an iWrite (precedent value overwritten) ...
[thread 0] Byte Value to be shifted at node 5 is 0x06
[thread 0] Calling an iApply ...
[thread 0] iApply finished at 1460539033.038021 s
[thread 0] Starting cycle 2 at 1460539033.038026 s
[thread 0] making an iWrite
[thread 0] Executed at 1460539033.038043 s
[thread 0] Making an iWrite (precedent value overwritten) ...
[thread 0] Byte Value to be shifted at node 5 is 0x07
[thread 0] Calling an iApply ...
[thread 0] iApply finished at 1460539033.061044 s
[thread 0] Starting cycle 3 at 1460539033.061052 s
[thread 0] making an iWrite
[thread 0] Executed at 1460539033.061080 s
```



[thread 0] Making an iWrite (precedent value overwritten) ...

[thread 0] Byte Value to be shifted at node 5 is 0x08

[thread 0] Calling an iApply ...

[thread 0] iApply finished at 1460539033.088264 s

[thread 0] end at 1460539033.088269 s

[thread 1] spawned at 1460539032.991799 s for node 7

[thread 1] Starting cycle 0 at 1460539032.991807 s

[thread 1] making an iWrite

[thread 1] Executed at 1460539032.991873 s

[thread 1] Making an iWrite (precedent value overwritten) ...

[thread 1] Byte Value to be shifted at node 7 is 0x07

[thread 1] Calling an iApply ...

[thread 1] iApply finished at 1460539033.013123 s

[thread 1] Starting cycle 1 at 1460539033.013131 s

[thread 1] making an iWrite

[thread 1] Executed at 1460539033.013161 s

[thread 1] Making an iWrite (precedent value overwritten) ...

[thread 1] Byte Value to be shifted at node 7 is 0x08

[thread 1] Calling an iApply ...

[thread 1] iApply finished at 1460539033.036111 s

[thread 1] Starting cycle 2 at 1460539033.036116 s

[thread 1] making an iWrite

[thread 1] Executed at 1460539033.036136 s

[thread 1] Making an iWrite (precedent value overwritten) ...

[thread 1] Byte Value to be shifted at node 7 is 0x09

[thread 1] Calling an iApply ...

[thread 1] iApply finished at 1460539033.059306 s



```
[thread 1] Starting cycle 3 at 1460539033.059311 s
[thread 1] making an iWrite
[thread 1] Executed at 1460539033.059330 s
[thread 1] Making an iWrite (precedent value overwritten) ...
[thread 1] Byte Value to be shifted at node 7 is 0x0A
[thread 1] Calling an iApply ...
[thread 1] iApply finished at 1460539033.085881 s
[thread 1] end at 1460539033.085886 s
```

We highlight in bold red font the log lines specifying the sequences to be written in the registers.

An example commented execution trace can be for instance:

```
SIR 8 TDI(02) ;
SDR 64 TDI(0707070707070707) ;
SIR 8 TDI(FF) ;
SDR 1 TDI(01) ;
SIR 8 TDI(01) ;
SDR 64 TDI(0505050505050505) ;
SIR 8 TDI(02) ;
SDR 64 TDI(0808080808080808) ;
SIR 8 TDI(FF) ;
SDR 1 TDI(01) ;
SIR 8 TDI(01) ;
SDR 64 TDI(0606060606060606) ;
SIR 8 TDI(02) ;
SDR 64 TDI(0909090909090909) ;
```



```
SIR 8 TDI (FF) ;  
SDR 1 TDI (01) ;  
  
SIR 8 TDI (01) ;  
  
SDR 64 TDI (0707070707070707) ;  
  
SIR 8 TDI (02) ;  
  
SDR 64 TDI (0A0A0A0A0A0A0A0A) ;  
  
SIR 8 TDI (FF) ;  
SDR 1 TDI (01) ;  
  
SIR 8 TDI (01) ;  
  
SDR 64 TDI (0808080808080808) ;
```

“SIR 8 TDI(01)” and “SIR 8 TDI(02)” set the IR register for selecting static_1 and static_2 respectively, while “SIR 8 TDI(FF)” selects the Bypass register. Execution is correct because each register is accessed 4 times with the right values shifted in. The bit sequence is also the same for each thread: in this case the hexadecimal representation is the same for both the Thread and SVF logs because the register size is a multiple of 4.

Please note that as the configuration algorithm is not optimal, there are several accesses to the “bypass” register, which are not useful. This is because in its current implementation, configuration aggressively deselects derivations which are not pending to reduce scan length to its minimum, even though this might result, as in this case, in additional “selection” steps.

NB: both the number of “static” Registers and the number of iterations of each thread can be customized by changing the appropriate “defines” at the beginning of the source file.

5. 1687 SEGMENT INSERTION BIT (SIB)

This use case represents the simplest usage of the 1687 SIB element, depicted in Figure 12: a 64-bit “static” register that is always in the scan path, and a 128-bit “dynamic” one whose inclusion is decided by the state of the SIB.

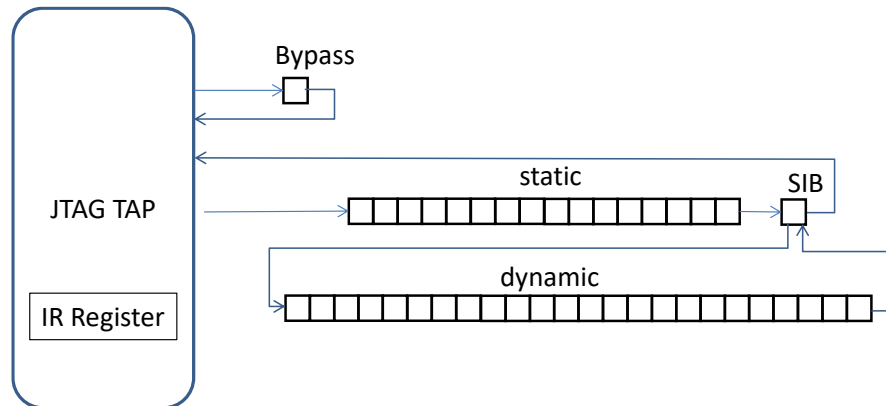


Figure 12 1687 Example

In MAST terms, its System Model is depicted in

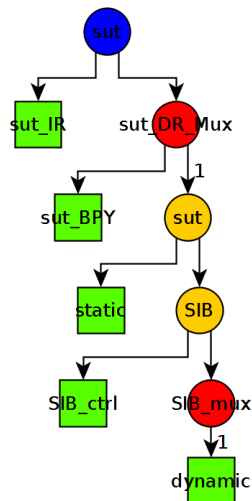


Figure 13 MAST System Model for Figure 11

The generic thread is launched on “static” with 4 accesses and “dynamic” with 1, resulting in the following results in terms of Thread execution and SVF commands:

```
[thread 0] spawned at 1460540230.775464 s for node 5
[thread 0] Starting cycle 0 at 1460540230.775477 s
[thread 0] making an iWrite
[thread 0] Executed at 1460540230.775532 s
[thread 0] Making an iWrite (precedent value overwritten) ...
```



```
[thread 0] Byte Value to be shifted at node 5 is 0x05

[thread 0] Calling an iApply ...

[thread 0] iApply finished at 1460540230.798130 s

[thread 0] Starting cycle 1 at 1460540230.798135 s

[thread 0] making an iWrite

[thread 0] Executed at 1460540230.798157 s

[thread 0] Making an iWrite (precedent value overwritten) ...

[thread 0] Byte Value to be shifted at node 5 is 0x06

[thread 0] Calling an iApply ...

[thread 0] iApply finished at 1460540230.824481 s

[thread 0] Starting cycle 2 at 1460540230.824487 s

[thread 0] making an iWrite

[thread 0] Executed at 1460540230.824511 s

[thread 0] Making an iWrite (precedent value overwritten) ...

[thread 0] Byte Value to be shifted at node 5 is 0x07

[thread 0] Calling an iApply ...

[thread 0] iApply finished at 1460540230.847273 s

[thread 0] Starting cycle 3 at 1460540230.847279 s

[thread 0] making an iWrite

[thread 0] Executed at 1460540230.847302 s

[thread 0] Making an iWrite (precedent value overwritten) ...

[thread 0] Byte Value to be shifted at node 5 is 0x08

[thread 0] Calling an iApply ...

[thread 0] iApply finished at 1460540230.872140 s

[thread 0] end at 1460540230.872146 s


[thread 1] spawned at 1460540230.775576 s for node 9

[thread 1] Starting cycle 0 at 1460540230.775582 s

[thread 1] making an iWrite
```

```
[thread 1] Executed at 1460540230.775608 s

[thread 1] Making an iWrite (precedent value overwritten) ...

[thread 1] Byte Value to be shifted at node 9 is 0x09

[thread 1] Calling an iApply ...

[thread 1] iApply finished at 1460540230.800085 s

[thread 1] end at 1460540230.800090 s


SIR 8 TDI(01);

SDR 65 TDI(000A0A0A0A0A0A0B);

SDR 193 TDI(000C0C0C0C0C0C0D09090909090909090909090909090909);

SDR 193 TDI(000C0C0C0C0C0C0C0C090909090909090909090909090909);

SDR 65 TDI(000E0E0E0E0E0E0E);

SDR 65 TDI(001010101010101010);
```

Please note the because of the SIB control register, the final vector size is not a multiple of 4, so hexadecimal representation is different for the “static” register. For instance, in the first SDR command the last byte is “0B”, which is the concatenation of “0000 0101 0000 0101” (0x0505) and “1” → “0000 0101 0000 0101 1” → “0 0000 1010 0000 1011” → “0A0B” in SVF terms where packing is done starting from the rightmost bit. This is highlighted by the light blue bold font in the SVF vectors.

NB: the size of the “static” and “dynamic” registers and the number of iterations of each thread can be customized by changing the appropriate “defines” at the beginning of the source file.

6. MULTIPLE INSERTION BIT (BIT)

This example depicts a MIB, the multi-bit extension of the SIB. The differences if compared with Figure 12 are:

- There are multiple “dynamic” registers;
- The control register is composed by multiple bits

- Many coding may be defined for segment selection
- Please note that MAST sees a SIB as a 1-bit MIB

Figure 14 shows the System Model for the MIB_testcase with default parameters: 4 dynamic segments and One-Hot coding, meaning the MIB_ctrl register has 4 bits.

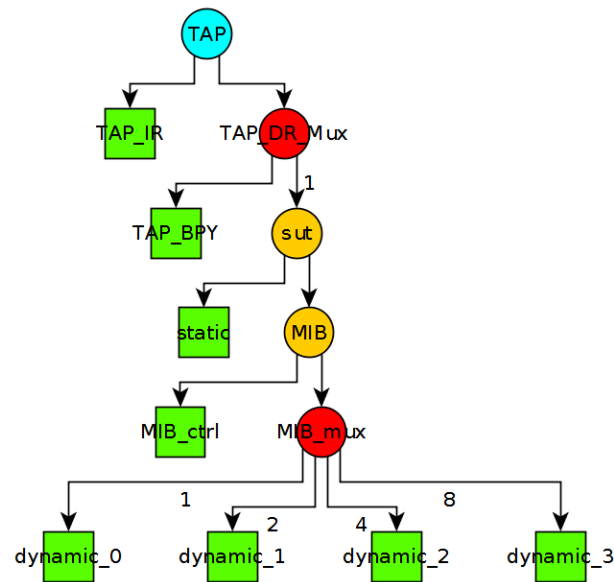


Figure 14 MAST System Model for the MIB example

The resulting SVF is:

```
SIR 8 TDI(01);

SDR 68 TDI(005050505050505058);

SDR 196 TDI(0060606060606060680C0C0C0C0C0C0C0C0C0C0C0C0C0C0C);

SDR 196 TDI(0060606060606060600C0C0C0C0C0C0C0C0C0C0C0C0C0C0C);

SDR 68 TDI(006060606060606064);

SDR 196 TDI(0060606060606060640B0B0B0B0B0B0B0B0B0B0B0B0B0B0B);

SDR 196 TDI(0060606060606060600B0B0B0B0B0B0B0B0B0B0B0B0B0B0B);

SDR 68 TDI(006060606060606062);

SDR 196 TDI(0060606060606060620A0A0A0A0A0A0A0A0A0A0A0A0A0A0A);

SDR 196 TDI(0060606060606060600A0A0A0A0A0A0A0A0A0A0A0A0A0A0A);

SDR 68 TDI(006060606060606061);
```




```
SDR 68 TDI (00606060606060606A);  
SDR 68 TDI (006060606060606062);  
SDR 193 TDI (000C0C0C0C0C0C0C0C0C0F0F0F0F0F0F0F0F0F0F0F0F0F0F);  
SDR 193 TDI (000C0C0C0C0C0C0C0C0C0D0F0F0F0F0F0F0F0F0F0F0F0F0F);  
SDR 68 TDI (006060606060606068); (5)  
SDR 68 TDI (006060606060606069);  
SDR 68 TDI (006060606060606061);  
SDR 193 TDI (000C0C0C0C0C0C0C0C0C0E0E0E0E0E0E0E0E0E0E0E0E0E);  
SDR 193 TDI (000C0C0C0C0C0C0C0C0C0D0E0E0E0E0E0E0E0E0E0E0E0E);  
SDR 68 TDI (007070707070707078);  
SDR 68 TDI (008080808080808080);
```

Patterns for the “dynamic” registers are highlighted in blue, while those for the “static” register are in red. The last SVF nibble is marked in green and can have different composition:

- (1) composed by the “SWIR” bit followed by the one-bit WBY register: $0x6 = 011\ 0 \rightarrow$ asking selection of the WIR, setting ‘0’ to WBY
- (2) Composed by the SWIR and the 3-bit WIR register: $0xB = 0x1\ 011 \rightarrow$ keep SWIR selected and put $0x011$ in WIR
- (3) Composed by the SWIR and the 3-bit WIR register: $0x8 = 0x0\ 011 \rightarrow$ deselect SWIR and put $0x011$ in WIR
- (4) Here the highlighted 8 bits are composed by the last 7 bits of “static” followed by the SWIR, left at ‘0’ in the case a) and put to ‘1’ in b) in order to access the WIR during the following cycle
- (5) These cycles are not useful and are generated by the non-optimal configuration algorithm which tries to completely close the wrapper by selecting the WBY (i.e. WIR set to “000”);

Please note that because of the non-optimal configuration algorithms SDR are done for each wrapped core, the second one being used to select the SWIR. An optimal scheduling would remove the first one as it is redundant.

8. MULTIPLE CHAINS

This example regroups the AccessInterface (4) and 1500 (7) examples: two 1500 wrappers on two different chains. It shows the capability of MAST of handling complex multi-chain topologies

and its aptitude for code reuse. The output of the example is too long to be commented here, and in any it is just a repetition of examples (4) and (7).

9. *DAISY CHAINING*

Since its very beginning, JTAG allowed TAPs to be concatenated one after the other, obtaining what is usually called a “daisy chain”, depicted in Figure 17.

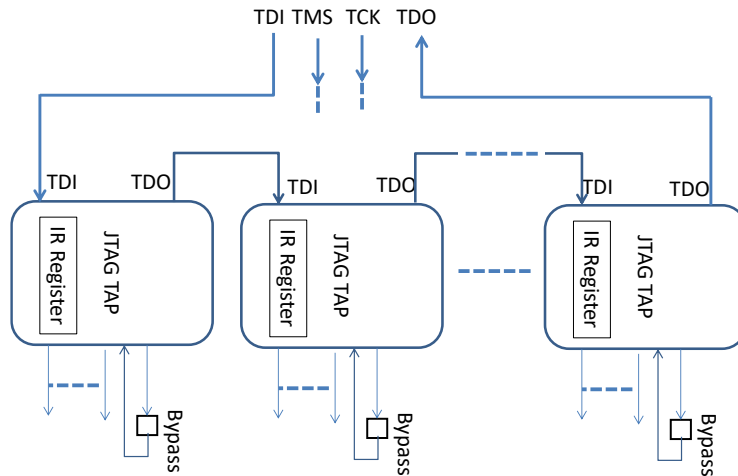


Figure 17 Daisy-chaining of JTAG TAPs

Even though extremely simple, some specific code was always necessary in ATPG to handle vector assembly and retargeting. 1687 did not solve the issue as the standard is defined only at the scan chain level. MAST on the other hand introduces the concept of AccessInterface: all TAPs are accessed through the same port, so they share the same AI. All it remains to do is express the sequence in the System Model, as shown in Figure 18, where we replicate 3 times the example of (4).

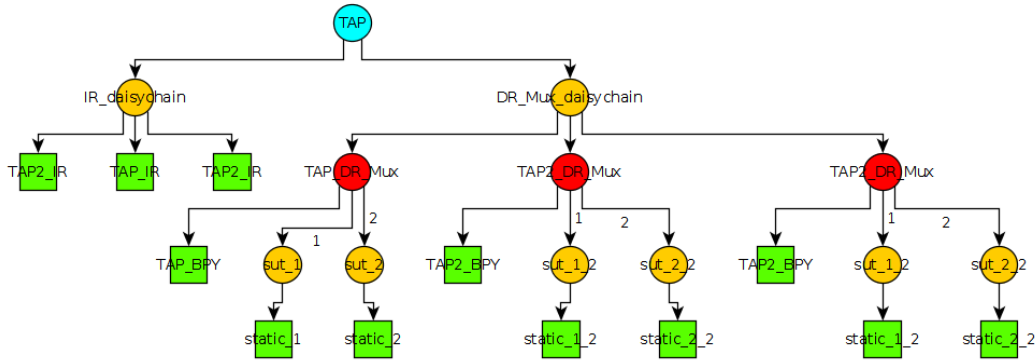


Figure 18 MAST System Model for a 3-TAP daisy chain

The resulting SVF execution is extremely simple to interpret:

```
SIR 24 TDI (020202);

SDR 192 TDI (07070707070707070F0F0F0F0F0F0F0F1919191919191919);

SIR 24 TDI (FFFFFF);

SDR 3 TDI (07);

SIR 24 TDI (010101);

SDR 192 TDI (05050505050505050D0D0D0D0D0D0D0D1717171717171717);

SIR 24 TDI (020202);

SDR 192 TDI (080808080808080810101010101010101A1A1A1A1A1A1A);

SIR 24 TDI (FFFFFF);

SDR 3 TDI (07);

SIR 24 TDI (010101);

SDR 192 TDI (06060606060606060E0E0E0E0E0E0E0E1818181818181818);

SIR 24 TDI (020202);

SDR 192 TDI (090909090909090911111111111111111B1B1B1B1B1B1B);

SIR 24 TDI (FFFFFF);

SDR 3 TDI (07);

SIR 24 TDI (010101);

SDR 192 TDI (07070707070707070F0F0F0F0F0F0F0F1919191919191919);

SIR 24 TDI (020202);
```

```
SDR 192 TDI (0A0A0A0A0A0A0A0A12121212121212121C1C1C1C1C1C1C);
SIR 24 TDI (FFFFFF);
SDR 3 TDI (07);
SIR 24 TDI (010101);
SDR 192 TDI (080808080808080810101010101010101A1A1A1A1A1A1A);
```

Each SIR command writes the daisy-chained IR register, composed of the concatenation of the 3 individual IR, while the following SDR command writes the vectors to the three concatenated scan chains. Note how MAST is able to handle this TAP-level topology without any case-specific code.

10. BSCAN2 BOUNDARY SCAN LINKER

The BSCAN2 Boundary Scan Linker from Lattice Semiconductors, whose block diagram is reproduced in Figure 19, is an excellent example of flow-breaking BS Linkers. To simplify, it is a “TAP of TAPs”: up to 4 TAPs can be attached, and the code word loaded into the “select” data register of the top-level BSCAN2 TAP decides which can be accessed or not. When multiple TAPs derivations (called “MSP”) are selected, they appear as in a daisy-chain with the BSCAN2 TAP itself.

The usage is complicated by the need of adding bypass registers in the “MSP” derivation to preserve timing.

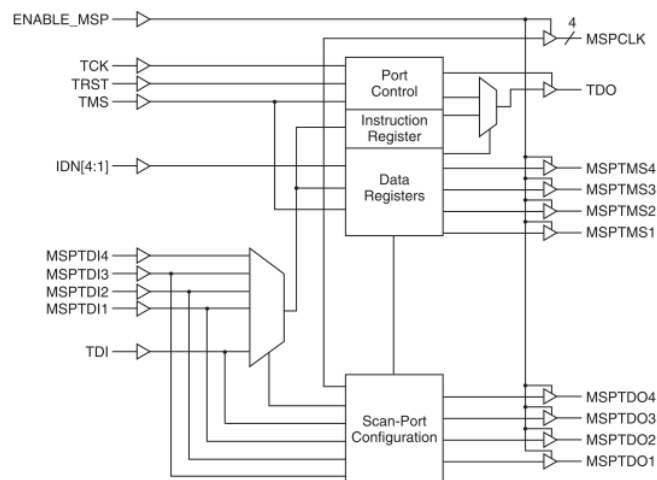


Figure 19 BSCAN2 linker, from Lattice documentation

In traditional JTAG, this behavior is extremely difficult to model and handle. In MAST, once more the AccessInterface abstraction helps because only the top-level TAP is actually connected to the outside and needs an AI, while the selection mechanism can be expressed in 1687 terms, as shown in Figure 20. The BSCAN2 top-level TAP is highlighted in the dashed blue line. Inside, the MSP selection is seen as two ScanMuxes on the IR and DR path whose control comes from the “Select” register. The three elements are highlighted by solid blue rectangles. Overloading of the “is_active/select/deselect” mechanism allow for an easy modeling. Red lines are used to highlight the first on the MSP TAP and its System Under Test: three registers connected in a chain, each one accessed by a thread.

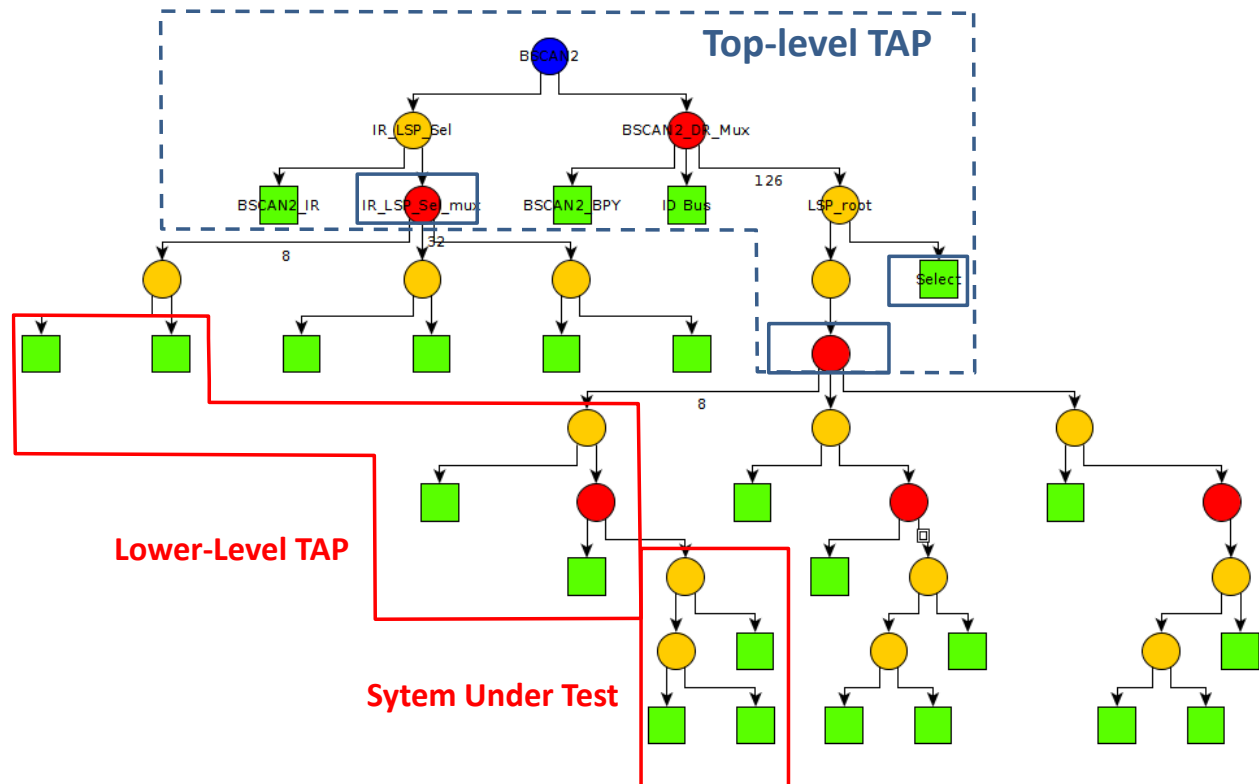


Figure 20 MAST System Model for the BSCAN2 Example

The resulting SVF commands are actually pretty simple:

```
SIR 8 TDI (7E); (1)
SDR 8 TDI (A8); (2)
SIR 35 TDI (00402017E); (3)
```

[illegible]

Vectors for the BSCAN2 TAP are highlighted in red, while those for the MSPs are in blue, green and orange. In details:

- (1) The BSCAN2 IR code word allowing access to the “Select” register is loaded
- (2) The code word allowing access to all MSP is loaded in the “Select” register
- (3) The daisy-chained MSP IR registers are set to select the DR registers. Please note that each IR is preceded by a bypass bit and that the BSCAN2 IR is the rightmost in the daisy-chaining, as per BSCAN2 requirements.
- (4) DR operation accessing all 9 registers. Please note that each DR is preceded by a bypass bit and that the BSCAN2 DR is the rightmost in the daisy-chaining, as per BSCAN2 requirements.

This example shows several features unique to MAST

- Handling of non-standard topologies in an unified way thanks to the “is_active/select/deselect” mechanism;
- Scheduling of several threads (9 here) regardless of their position in a complex topology
- Efficient configuration with no custom coding: MAST is able to find the optimal configuration where all MSPs are selected.

11. I2C ACCESS INTERFACE/ P1687.1

Thanks to its `AccessInterface` abstraction, MAST is able to generate operations to push/pull vectors on any arbitrary connection, not only JTAG. The new P1687.1 Working Group is trying to work in the same problem space, whence the name of the examples.

The first use case, `1687_1_testcase`, we replicate the 1500 example of (7), but instead of a top-level 1149.1 TAP we instantiate an I2C AccessInterface, configured as follows:

- Access to the IR chain is done by read/write to the register at address 24
- Access to the DR chain is done by read/write to the register at address 33

Structurally, the example is identical as in (7) as can be seen in Figure 21.

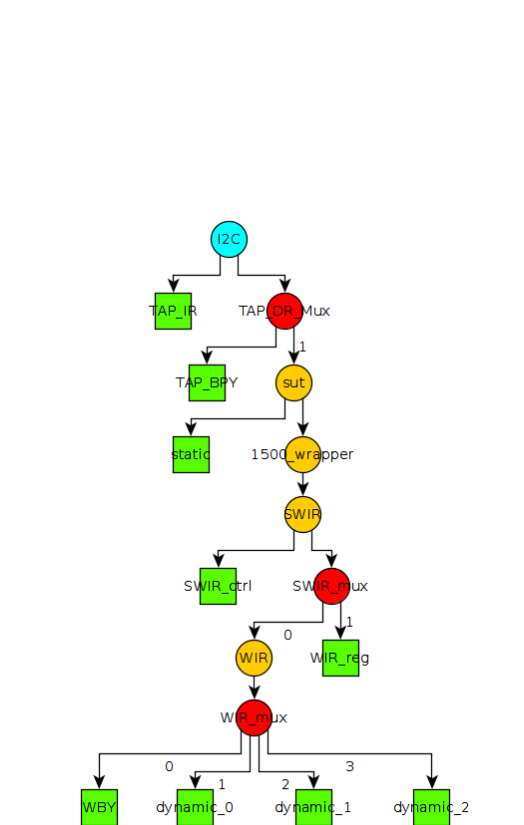


Figure 21 MAST System Model for the 1687.1 Example

Execution generates the following i2c operations:

```
i2c_read(24,1)

i2c_write(0x01,24,1)

i2c_read(33,9)

i2c_write(0x050505050505050580,33,9)

i2c_read(33,9)

i2c_write(0x0606060606060606B0,33,9)

i2c_read(33,9)

i2c_write(0x060606060606060630,33,9)

i2c_read(33,25)

i2c_write(0x06060606060606060808080808080808080808080800,33,25)

i2c_read(33,25)

i2c_write(0x06060606060606060688080808080808080808080800,33,25)

i2c_read(33,9)

i2c_write(0x060606060606060680,33,9)
```



```
i2c_read(33,9)

i2c_write(0x0606060606060606A0,33,9)

i2c_read(33,9)

i2c_write(0x060606060606060620,33,9)

i2c_read(33,25)

i2c_write(0x060606060606060607878787878787878787878787878780,33,25)

i2c_read(33,25)

i2c_write(0x060606060606060606878787878787878787878787878780,33,25)

i2c_read(33,9)

i2c_write(0x06060606060606060680,33,9)

i2c_read(33,9)

i2c_write(0x06060606060606060690,33,9)

i2c_read(33,9)

i2c_write(0x06060606060606060610,33,9)

i2c_read(33,25)

i2c_write(0x060606060606060607070707070707070707070707070700,33,25)

i2c_read(33,25)

i2c_write(0x060606060606060606870707070707070707070707070700,33,25)

i2c_read(33,9)

i2c_write(0x07070707070707070780,33,9)

i2c_read(33,9)

i2c_write(0x08080808080808080800,33,9)
```

Please note:

- Each SVF operation (SIR/SDR) is now replaced by a read+write operation having the same effect;
- SIR operations can be recognized by the access to register 24, SDR to register 33
- The vectors are the same as in (7). As I2C does not need a special byte packing like SVF, they are easier to interpret.

There is a second version of the example, `1687_1_BB_testcase`, which represents a hypothetical bit-banging interface: the I2C register has only 8 bits, each one representing a signal on the JTAG TAP. So, the user needs to drive the TAP by toggling the TCK and TMS signals through consecutive I2C operations. In MAST this can be represented by a different implementation of the `AccessInterface` action, but with no modification to either the System Model or the core MAST code. The complete output is too long to be copied here.

These two examples validate the `AccessInterface` abstraction, and show the capability of MAST to support any arbitrary interface and to be “1687.1 Compliant” even before the standard is released.

12. MIXED I2C/JTAG ACCESS LINK (BROCADE)

JTAG is by definition a closed world: ATPG and EDA tools can work thanks to the standardized hardware and descriptions, but in the meantime anything that cannot fall into these categories cannot be handled. Although not a big problem in chip-level testing, where the designer has complete control over the device, this can be a major issue in board and system-level where different approaches might coexist. Other communication standards like I2C or SPI are widely used to access chips and configure the board, for instance for Power-Up. An archetypical example is the JTAG Mux developed by Brocade and depicted in Figure 22 : the component is a Boundary Scan linker pretty much like the BSCAN2, but the active JTAG TAPs are decided by the register marked as “GPIO Header”, which must be accessed through I2C.

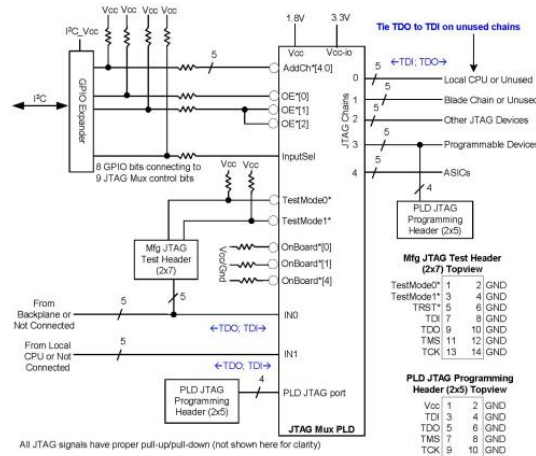


Figure 22 Brocade JTAG Mux (from the datasheet)

This means that the ATPG cannot take care of its configuration: a higher level is required to post-process the ATPG vectors and take care of the non-JTAG operations. On the other hand MAST can exploit the AccessInterface abstraction to represent in the System Model, as shown in Figure 23: the GPIO header, called “Brocade_CTRL_reg” is in the left side of the picture, and it is connected to an I2C Access Interface. On the right side, access to the JTAG TAPs (composed for simplicity by only one chain) is handled by two ScanMuxes, one for IR and one for DR. Both are controlled by the “Brocade_CTRL_reg”, as highlighted by the dashed arrows.

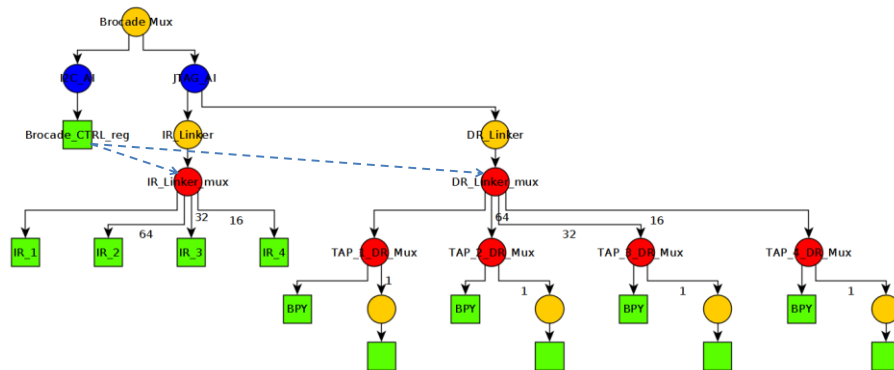


Figure 23 MAST system model for the Brocade Mux

The interest of this example is that thanks to the AI abstraction we can express in 1687 terms (i.e. a register controlling two ScanMuxes) a dependency outside of JTAG. As a result, the MAST



execution engine can directly handle the configuration of the system and generate the mixed I2C/JTAG transaction, copied here:

```
i2c_read(29,8)

i2c_write(0xF0,29,8)

SIR 32 TDI(01010101);

SDR 256 TDI(0D0D0D0D0D0D0D0D131313131313131319191919191919191F1F1F1F1F1F1F1F);
```

Please note how MAST is able to compute the optimal configuration of the “Brocade_CTRL_reg : 0xF0, where all four TAPs are selected.

References

- [1] Portolan M., “Automated Test Flow: the Present and the Future”, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (IEEE-TCAD), December 2019, DOI:10.1109/TCAD.2019.2961328
- [2] Portolan Michele, “A novel test generation and application flow for functional access to IEEE 1687 instruments”, Test Symposium (ETS), 2016 21th IEEE European, 23-27 May 2016, DOI: [10.1109/ETS.2016.7519302](https://doi.org/10.1109/ETS.2016.7519302)